

AI Usage report
Advanced Programming

Lorenzo Ceccotti 635200

July 23, 2026

Contents

1	Introduction	2
1.1	What was built	2
1.2	AI models used	2
1.3	AI method of usage	3
2	Background	4
2.1	Markdown parser architecture	4
2.2	C++ constructs and design patterns	4
2.3	Mermaid support	4
3	AI Usage	5
3.1	Learn	5
3.2	Plan and Implement	5
3.3	Correct	6
3.4	Verification	6
3.5	Conclusion	7
A	Relevant Chats	8
A.1	Learn: early design and C++ research	8
A.2	Plan and implement	8
A.3	Correct	9
A.4	Verification, memory and performance	9
A.5	Commits without a recorded chat/prompt	9

Introduction

This report explains what was built and how AI was used to build it. The whole project was developed in C++, except for a few utilities where JavaScript was unavoidable. The choice of C++ was driven mainly by the goal of learning new language constructs and revising some design patterns; it also helps that C++ is a language LLMs know well.

1.1 What was built

The project produced a Markdown parser, written in C++, that implements the CommonMark v0.31.2 specification: headings, lists, links, emphasis, code blocks, and the rest of the standard features. On top of that, ````mermaid` fenced blocks are detected and tagged as separate diagram nodes. From the same parse tree the project can emit two different outputs, HTML and a JSON AST that follows the *mdast* format used by the remark/unified ecosystem.

For Mermaid the work does not stop at tagging the blocks. The project also includes a native interpreter for the flowchart/graph diagram type that takes the diagram text and produces an SVG on its own, without a browser and without the Mermaid JavaScript library. This interpreter relies on a small LR(1) parser generator built as a separate tool, plus a layout engine and an SVG renderer.

Finally, the whole parser is compiled to WebAssembly and runs in a live two-pane web editor, so the same C++ code that runs on the command line also runs in the browser. The design choices behind these programs are summarised in the Background chapter, with the full technical documentation in the project's docs/ folder.

1.2 AI models used

The entire project was carried out on a Claude Pro subscription, so token usage was never a constraint. This would also have allowed a higher thinking effort, but the models were all kept on their **medium** setting rather than a higher one. The subscription also includes Claude Code, which was used through its **VS Code** extension.

From the point of view of the models involved, the project divides into two parts. The first part relied on Claude Sonnet 4.6 and covers:

- **Learning:** questions about Markdown parsers, Mermaid, WebAssembly, and C++ constructs.
- **Initial Markdown parser implementation:** planning and development.
- **Iterative parser bug fixes:** evaluation and verification of the AI output.

The second part switched to a more capable model, Claude Opus 4.8. This one was used for brainstorming and implementing the possible refactors, and for the design and implementation of the Mermaid interpreter.

1.3 AI method of usage

Both Claude Sonnet 4.6 and Opus 4.8 were used to ship features by following the same workflow:

- **Learn:** understand the topic behind the feature to integrate.
- **Design:** brainstorm and design a solution that integrates it.
- **Testing:** set up a way to test the AI output.
- **Implement:** define the specification that constrains what the AI is going to write.
- **Evaluation and Validation:** review the code and use the testing method set up earlier to assert the functional requirements.
- **Bug fixing:** iteratively correct the AI to fix bugs or poorly written code.

This workflow made it possible to keep control over the codebase even when delegating the implementation to Claude Code. One point became clear after switching to Opus 4.8: the model already arrived with its own plan for each feature. More capable models like Opus 4.8 seem to aim for “one-prompt” features, delivering a whole feature in a single shot. The AI Usage chapter follows these workflow stages and shows the kind of prompts used at each of them.

Background

This chapter gives just enough technical context to follow how AI was used in the rest of the report. The full design is documented in the project’s docs/ folder, which is referenced throughout; the summaries here are deliberately short.

2.1 Markdown parser architecture

Markdown is not a clean context-free language — whether a line continues or opens a block depends on indentation and on which blocks are currently open — so the parser follows the two-phase, line-based algorithm of the CommonMark reference implementation rather than a grammar. Phase 1 maintains a *spine* of open blocks and, for each line, decides what it continues, opens, or closes; phase 2 parses the inline content of each leaf block. The details are in docs/05_spine_handler.md, docs/03_continuation_rules.md, and docs/08_data_flow.md.

The tree is *render-neutral*: a single AST is emitted as both HTML and a JSON AST, with every format-specific normalisation kept in the renderers rather than the parser (docs/12_renderers.md, docs/index.md). This also works as a correctness check — a format decision that leaks into the parser shows up as a failure in the other output’s tests. Ownership of the tree is single, through `std::unique_ptr`; its one real consequence is that destroying a very deep tree can overflow the stack, which is handled by a nesting cap.

2.2 C++ constructs and design patterns

One stated goal was to learn and exercise C++, so the codebase uses a number of constructs, each researched before use (Chapter 3): `enum class` together with `std::variant` and `std::visit` for node payloads; `std::unique_ptr` and `std::move` for ownership; `std::string_view` for zero-copy line slicing; a C++20 concept (`Renderer`) that lets `parse()` accept any output format without changes; and the Visitor, Factory, and Meyers-singleton patterns for rendering and for fenced-block handling. The project layout, the CMake build with `FetchContent`, and the build-time LR(1) parser generator are described in docs/01_project_structure.md and docs/02_data_types.md.

2.3 Mermaid support

Mermaid is a family of diagram types; the project targets one, the flowchart/graph type at Mermaid 11.0.0 syntax, chosen because it involves both a real grammar and a graph-layout problem. It is handled as a pipeline — lexer, generated LR(1) parser, `lower()` to a `FlowDb` description, a hand-written minimal Sugiyama layout, then SVG — with no external graph library. The layout needs to know how wide each label is, so measurement sits behind a `TextMeasurer` interface: an approximation on the command line, and the browser’s real `measureText` in the WebAssembly build. The full design, and the features intentionally left out, are in docs/mermaid/.

AI Usage

This chapter aims to explain how AI was actually used to build the project, following the workflow from the Introduction. Each section shows a few prompts rather than the whole prompt list (which is submitted alongside this report in the `prompts/` folder), because for how the AI was used the prompts alone, without the context around them, are misleading and say little on their own. Another thing to point out is that some of the chats were lost along the way, so a good part of the bug fixing and a few of the refactors are recorded only by their git commits, which are still the result of AI assistance even without a saved conversation behind them; the commits with no matching chat are pointed out in the appendix.

3.1 Learn

Every feature started by using the AI to understand the topic (when needed) before writing any code; there was also a learning and question session to understand the code the AI would then write, so as to suggest insights and keep control of what was going on. The questions were of two kinds. Some were about the problem domain, for example whether a grammar-based parser even fitted Markdown:

“Is .md a context sensitive language or is it context free one?”

The answer — that CommonMark defines an algorithm rather than a grammar — is what led to the spine design instead of an LR/LL parser. Others were about a specific C++ construct that a later part of the code would use:

“What are variants, what does it mean type safe union?” [...] “What is `std::string_view`?”

In each case the construct was understood on its own first and only then used.

3.2 Plan and Implement

A feature was not handed over as a single vague request. The pattern was to design a solution, agree on it, then write a specification tight enough to constrain the output, and only then let the model implement. The clearest example is the parser architecture, produced as a written design document before any code. The same habit applied to the harder Mermaid parser generator, where the model was made to explain itself before writing anything:

“Before implementing the generator explain to me the steps you are going to do and describe the generator structure plus the final structure of the generated parser.”

Only after that discussion was the generator implemented (commit 4ab3331), followed by the Mermaid parser and the lowering stage (commit c4473cb). Fixing the plan in advance is what kept the output reviewable against an agreed target rather than accepted on trust.

3.3 Correct

Having a good knowledge of the topic is what made it possible to question the model on what it was doing, and to redirect it when it went the wrong way. This is also where a difference between the models showed up: a capable model like Opus 4.8 questions the user on some design choices and quietly decides others on its own, so blindly following it gives the feeling of understanding the problem while the model is the one doing the real work.

The corrections were of two kinds. Sometimes it meant overriding a design the model proposed, like when it wanted to keep a hand-written parser on top of the generated tables:

“The parser has to be generated by the tool, it does not make sense that I have an hand-written parser when I’m automating the parser.”

Other times the correction was more structural, like noticing that the parser had taken on work that belonged to the renderers, and moving that normalisation back downstream (commit f2e0cf3).

3.4 Verification

Verification is what the assignment asks the report to focus on, and the project was built to make it cheap. The principle was to check every AI-written component against an external reference it could not have reproduced from memory, and this was scoped early:

“How can I verify the correctness of what my parser output?”

Verification is the most important part of the whole development, it was the way to make sure the parsers’ output was tested correctly, both for Markdown and for Mermaid, and it is what allows the iterative bug fixing, setting up de facto a test-driven development.

For Markdown the test file provided by commonmark.org was used, it comprises more than 600 tests and goes over every feature of standard Markdown. The way to cover that file was itself brought up as a design question, together with the wish to see, for every failing case, what was expected against what was actually produced:

“For my markdown parser I have a single file provided by CommonMark which consists in a long list of test cases, what is the best approach to cover it? I would like to know for each test if it passed or not and if not what was the expected and what instead was the actual.”

The result is a test suite that runs every example and, when the output does not match, prints the expected and the actual HTML side by side, which is what made fixing the many edge cases manageable.

The same parse tree is also emitted as JSON, following the mdast format, and this second output is verified in the same way, this time against trees generated by remark, the reference implementation of the format. Because both outputs come from one tree, a decision that satisfies one reference but not the other is caught immediately, and adding this second check was scoped as:

“I think we should add tests also for the json output, how would you proceed?”

The reference trees are generated once and then compared against, after normalising both sides so that the comparison is structural rather than byte-for-byte (commits 4dfd2cc, 5228dc0, f42de5e).

For Mermaid it was harder to set up the test suite, because the output is an SVG and cannot be compared pixel by pixel, since the same diagram can be drawn correctly in many slightly different ways. The output was therefore checked structurally instead, by generating the reference from the official Mermaid JavaScript library and comparing the parser’s own syntax tree against it, so that what is verified is the structure of the diagram and not its exact drawing; on top of that, unit tests were added for the lexer, the parser and the lowering stage. Where and against what to verify was discussed before any of it was written:

“Verification, where you think it is better to verify the mermaid output? [...] use mermaid.js [...] and use its output against mine [...] Also I would like to check the AST structure, does mermaid.js outputs the AST instead of the SVG?”

and the strategy was brought to the model already formed, to be reviewed rather than invented:

“I already setup the verification strategy which is the following: [...] run `extract.mjs` on them to generate the SVG and the AST in json format [...] then check our mermaid parser against the pre-generated SVG and AST. Review this strategy.”

The `lower()` output is then compared against that AST across the whole fixture corpus (commits 9dff979, 106d944).

3.5 Conclusion

Two things stood out from working this way. First, the more capable model, Opus 4.8, tends to arrive with a complete plan and to deliver the whole feature from a single prompt. Second, staying in charge depends on two things: having a strong knowledge of the programming language, and having a strong understanding of the direction the feature is going once a solution has been planned. Without both, the model is the one deciding what gets written, not the developer.

Relevant Chats

This appendix names the conversations most relevant to the report, grouped as in the AI Usage chapter, each with the file that holds it. The early design work was done in the `claude.ai` web interface (`prompts/web/`); the rest in Claude Code (`prompts/`).

A.1 Learn: early design and C++ research

- **What is Markdown** — context-free versus context-sensitive, the cmark parsing strategy, and lazy continuation. `prompts/web/01-what-is-markdown-md.md`
- **Smart pointers in C++** — smart pointers, templates, variants, `std::move`, `std::string_view`. `prompts/web/03-smart-pointers-in-cpp.md`
- **C++ project structure and file conventions** — headers and sources, namespaces, CMake, testing. `prompts/web/04-cpp-project-structure-and-file-conventions.md`
- **Mermaid block types** — diagram types, how Mermaid produces an SVG, and its layout strategy. `prompts/web/13-mermaid-block-types.md`

A.2 Plan and implement

- **Markdown parser architecture and component design** — the component design and data flow, written before coding. `prompts/web/05-markdown-parser-architecture-and-component-design.md`
- **Mermaid rendering approach** — browser versus native rendering, and why a layout engine is needed. `prompts/01-mermaid-rendering-approach.md`
- **Fence handlers, factory and singleton** — specialising fenced blocks through a handler registry. `prompts/02-fence-handlers-factory-singleton.md`
- **Renderer concept and templates** — templating `parse()` and defining the `Renderer` concept. `prompts/04-renderer-concept-and-templates.md`
- **Mermaid native parsing pivot** — the decision to parse Mermaid in C++. `prompts/06-mermaid-native-parsing-pivot.md`
- **Grammar and generator design** — the flowchart grammar and the decision to generate the parser. `prompts/07-grammar-and-generator-design.md`
- **LR(1) generator implementation** — LR items, and LALR versus canonical LR(1). `prompts/08-lr1-generator-implementation.md`
- **Mermaid AST and lowering** — the AST types and the lowering to `FlowDb`. `prompts/09-mermaid-ast-and-lowering.md`

- **Layout (Sugiyama)** — the four-phase layout and the dual-build measurement idea. `prompts/11-layout-sugiyama.md`
- **WASM integration and text measurement** — `emscripten::val` and the browser measurer. `prompts/12-wasm-integration-and-text-measurement.md`

A.3 Correct

- **JSON output and mdast alignment** — making the `--json` output a real, specified format. `prompts/15-json-output-and-mdast-alignment.md`
- **Link definitions and references** — the Definition node design that the model defended. `prompts/16-link-definitions-and-references.md`
- **Normalisation: parser versus renderer** — decoupling the parser from the renderer. `prompts/17-normalization-parser-vs-renderer.md`

A.4 Verification, memory and performance

- **Golden verification vs Mermaid** — verifying the AST against the official library. `prompts/10-golden-verification-vs-mermaid.md`
- **Memory ownership and leak investigation** — checking whether a leak was even possible. `prompts/19-memory-ownership-and-leak-investigation.md`
- **Recursive destructor and nesting caps** — bounding nesting to avoid a stack overflow. `prompts/21-recursive-destructor-and-nesting-caps.md`

A.5 Commits without a recorded chat/prompt

Not every commit has a conversation behind it. Some of the chats, mostly from the early and most intensive bug fixing of the markdown parser, were lost, so a part of that work survives only as git commits. Those commits were made with AI assistance just like the rest, and those chats were also the most predictable ones, since they were mostly about fixing the code step by step to make each test pass, triggering a refactor when needed.

Parser bug fixes — setext promotion, code fences, entity and backslash decoding, UTF-8, inline delimiter stack, line breaks, raw HTML, image and URL escaping: 74a1265, 887dfdc, 8e72d3b, ff1711e, 63e88bc, 5df03cd, 7e96484, 254950c, a284ef5, 918c1b1, 547d6f8, 506e517, fc02916, 740646d, 8fe4917, 378f572, 51a7f22, 85fc6dd, 5f04572.

Early parser refactors — ScannedLine and PreScanner cleanups, removing magic numbers, moving utilities out of classes: 474c59e, 2767375, 2ca222d, c7e665f, 1e25aa6, 3ac6500.